Radiant College, Manigram Butwal

[ITEX 103] Object Oriented Programming

B Tech Ed. – 2ND SEMESTER

Unit – 6

(Templates)

Objective:

1. **Concept of templates**
2. **Class templates**
3. **Function Templates**
4. **Concept of Frameworks**

Templates

Definition: Templates allow writing generic code that can work with any data type, reducing redundancy and improving reusability.

Purpose: Enable the creation of functions and classes that can operate on various types without explicit type specification.

Syntax: Uses the template keyword followed by type parameters (e.g., T).

Types:

- **Function Templates**: Generic functions.
- **Class Templates**: Generic classes.

Advantages:

- Code reusability.
- Type safety at compile time.
- No runtime overhead for type casting

Example:

```
template <typename T>
T add(T a, T b) {
    return a + b;
}
// Usage: int sum = add<int>(2, 3);
```

Class Templates

Definition: A blueprint for creating classes that can handle different data types.

Syntax: Define a class with a template parameter.

Example:

```
#include <iostream>
using namespace std;

template <typename T>
class Pair {
private:
    T first;
    T second;
public:
    Pair(T f, T s) : first(f), second(s) {}
    T getFirst() const { return first; }
    T getSecond() const { return second; }
};

int main() {
    Pair<int> intPair(1, 2);
    cout << "First: " << intPair.getFirst() << ", Second: " <<
intPair.getSecond() << endl;
    return 0;
}
```

Key Points:

- Instantiated at compile time based on the type provided.
- Can have multiple template parameters (e.g., template <typename T, typename U>).
- Useful for containers like stacks or pairs.

Function Templates

Definition: Functions that can operate on multiple data types without rewriting code.

Syntax: Use template <typename T> before the function declaration.

Example:

```
#include <iostream>
using namespace std;

template <typename T>
T maxValue(T a, T b) {
    return (a > b) ? a : b;
}

int main() {
    cout << "Max (int): " << maxValue(5, 10) << endl;
    cout << "Max (double): " << maxValue(3.14, 2.71) << endl;
    return 0;
}
```

Key Points:

- Automatically generates type-specific versions at compile time.
- Can be overloaded or specialized (e.g., template <> void maxValue<int>(int a, int b)).
- Default arguments can be used with template parameters.

Concept of Frameworks

Definition: A framework is a reusable set of libraries or tools providing a structure for developing applications, often built using templates for flexibility.

Relation to Templates: Templates are foundational in frameworks like the Standard Template Library (STL) in C++ (e.g., vector, map).

Characteristics:

- Predefined structure and components.
- Inversion of control: The framework calls your code (e.g., callbacks, hooks).
- Extensibility through templates.

Examples:

- **STL**: Uses templates for generic containers and algorithms.
- **Boost**: A collection of template-based libraries for advanced C++ features.
- **Qt**: A framework with templated signal-slot mechanisms.

Advantages:

- Speeds up development.
- Ensures consistency and best practices.
- Supports scalability with generic programming.

Example

```
#include <vector>
using namespace std;

int main() {
    vector<int> vec = {1, 2, 3, 4};
    for (int num : vec) {
        cout << num << " ";
    }
    cout << endl;
    return 0;
}
```